TECHNICAL AUDIT & REMEDIATION PLAN

OXENN

Codebase Integrity Audit Report

An evidence-based audit of the Oxenn repository covering dependencies, the Drizzle database layer, configuration and build tooling, and application security. Every finding is backed by file-level citations and verified tool output, paired with a phased remediation roadmap and a command-level execution checklist.

Repository	github.com/DiabloNova/oxenn
Stack	Next.js 16 · Drizzle ORM · PostgreSQL · pnpm
Prepared for	Oxenn Engineering Team

OCTOBER 2026 · DEEP-DIVE EDITION

Table of Contents

1. Executive Summary . . . 3

2. Audit Scope and Methodology . . . 5

2.1 Scope . . . 5

2.2 Method and evidence discipline . . . 5

2.3 Severity model . . . 6

3. System Overview and Architecture . . . 6

3.1 Stack and structure . . . 6

3.2 Tenant isolation design . . . 6

3.3 Migration chain inventory . . . 7

4. Critical Findings . . . 7

C1 — Next.js 16.2.11: three CRITICAL RCE advisories, fix requires >= 16.3.6 . . . 7

5. High-Severity Findings . . . 8

H1 — Stored XSS: unsanitized LLM output injected via dangerouslySetInnerHTML . . . 8

H2 — undici 7.29.0: ten advisories on the AI runtime path . . . 9

H3 — axios < 1.20.0: twelve advisories via the crawler dependency . . . 9

H4 — sharp < 0.35.4: libvips and libheif CVEs via Next.js . . . 9

H5 — eslint 9.39.5 deprecated and end-of-life . . . 9

H6 — js-yaml < 4.3.2 and @grpc/grpc-js < 1.14.5 (transitive HIGHs) . . . 10

6. Medium-Severity Findings . . . 10

M1 — @types/node ^20 against engines node >=22 and CI Node 22 . . . 10

M2 — @types/dompurify deprecated stub; dompurify below patched floor . . . 10

M3 — drizzle-kit 0.31.10 pulls the deprecated @esbuild-kit chain . . . 10

M4 — Mixed package-manager invocations inside a pinned-pnpm project . . . 11

M5 — pnpm 10 blocked build scripts are undeclared . . . 11

M6 — .env.example drift and weak env-check coverage . . . 11

M7 — Broken shadcn config and orphan root component trees . . . 11

M8 — next.config.ts empty: no security headers, no hardening . . . 12

M9 — Test coverage gaps and warning-tolerant gates . . . 12

7. Low-Severity and Hygiene Findings . . . 12

L1 — Committed junk and stale artifacts . . . 12

L2 — Migration file lacks trailing newline . . . 12
L3 — Inngest scaffolding and rename leftovers . . . 13
L4 — tsconfig target ES2017 for a Node >=22 codebase . . . 13
L5 — Audit-log records carry placeholder identity fields . . . 13
L6 — pgvector extension not encoded in the migration chain . . . 13
8. Verified Strengths (Evidence-Based) . . . 14
9. Execution Plan — Phased Roadmap . . . 15
Phase 0 — Safety net (before any change) . . . 15
Phase 1 — Dependency surgery (C1, H2-H6, M1-M3, M5) . . . 15
Phase 2 — Code security (H1, M8) . . . 16
Phase 3 — Repository and configuration hygiene (M6, M7, M9, L1, L2, L4) . . . 16
Phase 4 — Scheduled follow-ups (separate change window) . . . 16
9.3 Risk assessment of the plan . . . 16
10. Command-Level Checklist . . . 17
Appendix A — Tool Output Evidence . . . 18
A.1 pnpm audit summary (2026-10-19) . . . 18
A.2 Key lockfile deprecation records (verbatim) . . . 18
A.3 Test and build evidence . . . 19
A.4 Environment facts . . . 19
Appendix B — Dependency Upgrade Matrix . . . 19

1. Executive Summary

This report presents a complete, evidence-based audit of the **Oxenn** repository (github.com/DiabloNova/oxenn), analyzed from the uploaded snapshot **oxenn-main (2).zip** (769 files). The application is a Next.js 16.2.11 / React 19.2.4 / TypeScript 5.9.3 multi-tenant SaaS product backed by PostgreSQL with pgvector, Drizzle ORM 0.45.2, drizzle-kit 0.31.10 and pnpm 10.30.3. Every finding in this document is backed by concrete evidence: file paths with line numbers, code excerpts, lockfile entries, or tool output captured by running the project's own toolchain end-to-end in a clean environment.

The headline conclusion is nuanced. The parts of the repository the user suspected most — the Drizzle schema and the migration chain — turned out to be structurally **sound**: the journal and snapshot chain 0000-0012 passes **drizzle-kit check**, all 51 tables that receive FORCE ROW LEVEL SECURITY in migration 0005 have a complete set of tenant-isolation policies created in migration 0000, and the migrator correctly bootstraps the pgvector extension before applying migrations. The genuinely urgent problems live elsewhere: the dependency tree carries **three CRITICAL remote-code-execution advisories against the pinned Next.js 16.2.11** (fixed only in 16.3.6+), 25 HIGH advisories concentrated in transitive packages (undici, axios, sharp), and one **confirmed stored-XSS sink** in the LLM analytics panel where unescaped model output reaches dangerouslySetInnerHTML.

3 CRITICAL

Next.js RCE advisories — next 16.2.11, patched in >= 16.3.6

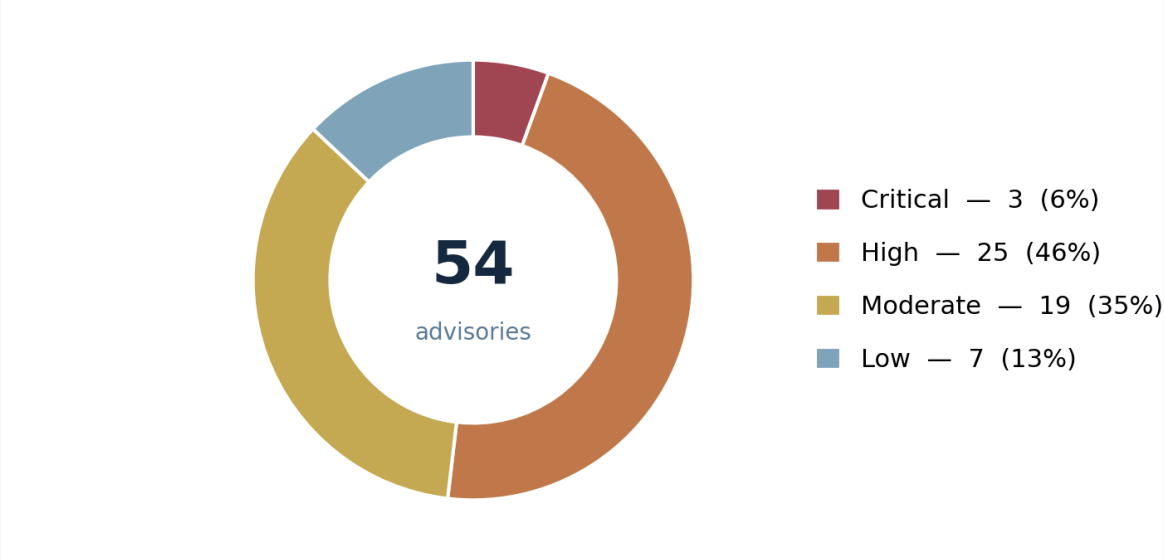


Figure 1 — pnpm advisory severity distribution (54 advisories, 2026-10-19).

Overall health is good for a project at version 0.1.0. The toolchain verification matrix below was produced by actually executing each command; nine of eleven checks pass outright. The two failures are security-scan categories (dependency CVEs and the XSS sink), not build or correctness failures. This distinction drives the remediation strategy in Chapter 9: the project does not need stabilization work first, so the plan can move directly to dependency surgery and security fixes behind the existing, already-strong CI gates.

Verification matrix (executed 2026-10-19)

Check	Command	Result
Lockfile integrity	pnpm install --frozen-lockfile	PASS (11.4 s, no drift)
Type safety	pnpm typecheck (tsc --noEmit)	PASS (0 errors)
Migration chain	npx drizzle-kit check	PASS (13 entries consistent)
RLS migration coverage	pnpm db:verify-rls-migrations	PASS
Unit tests	pnpm test:unit	PASS (164/164, 41 files)
UI tests	pnpm test:ui	PASS (3/3, 1 file)
Production build	pnpm build	PASS (full route manifest)
Lint discipline	pnpm lint	WARN (0 errors, 218 warnings)
Dependency CVEs	pnpm audit	FAIL (54 advisories, 3 critical)
XSS surface	manual render-path review	FAIL (1 unsanitized sink)
Secrets scan	key-pattern grep, all sources	PASS (none found)

Top risks, in priority order: **(1)** Next.js RCE class vulnerabilities — unauthenticated code execution vectors in the dev server (Windows), the Image Optimization pipeline, and next/og ImageResponse; **(2)** a stored XSS in src/components/features/analytics/LlmAnalyticsPanel.tsx where LLM output reflecting crawled third-party web content is rendered raw into the authenticated dashboard; **(3)** a transitive supply-chain backlog — undici 7.29.0 alone carries ten advisories including a TLS certificate validation bypass on a runtime path (AI provider calls). None of these require schema or data migration changes, which keeps the blast radius of the fix small and fully reversible.

A companion remediation kit (folder oxenn-fixes/) ships alongside this report containing a corrected package.json (version bumps, pnpm overrides, build-script allowlist, script hygiene), a hardened next.config.ts with baseline security headers, a corrected components.json, a new src/lib/utils.ts, a whitespace-corrected 0004 migration file, an updated .env.example covering previously undocumented variables, and a step-by-step patch guide for the XSS fix plus a documented deletion list for repository junk files.

2. Audit Scope and Methodology

2.1 Scope

Four domains were audited, matching the agreed engagement scope: **dependencies** (manifest, lockfile, registry state, peer relationships, deprecation and advisory status), **database and Drizzle** (schema, migration SQL, journal, snapshots, migrator, guard scripts, RLS design), **configuration and build** (tsconfig, next.config, eslint, vitest, CI workflows, root-level scripts), and **security and code** (injection sinks, sanitization coverage, secrets, env handling, tenant isolation). The source of truth was the uploaded zip; registry facts were verified live against the npm registry on 2026-10-19.

2.2 Method and evidence discipline

The audit follows a strict evidence hierarchy. First, static reading of configuration and code with exact file/line citations. Second, cross-checking claims against the lockfile (pnpm-lock.yaml, format 9.0, 2,869 packages) rather than trusting manifest ranges. Third, dynamic verification: the full toolchain was executed — pnpm install --frozen-lockfile, tsc --noEmit, eslint, drizzle-kit check, vitest unit and UI projects, the production build, and pnpm audit --json. Fourth, hypothesis falsification: two early hypotheses were explicitly tested and **rejected** when the evidence disagreed, and are documented here so the reader knows what was ruled out and why.

- **Ruled out — empty migration 0004.** wc -l reported 0 lines for database/drizzle/0004_abandoned_black_crow.sql. Byte-level inspection (wc -c = 117, cat -A) proved the file contains a valid CREATE INDEX (HNSW, vector_cosine_ops) statement with no trailing newline; wc -l counts newlines, not content. The snapshot-to-SQL chain is consistent.
- **Ruled out — missing 0005 snapshot as corruption.** meta/ has no 0005_snapshot.json, which initially suggests journal drift. Migration 0005 begins with the header “Custom SQL migration file” — drizzle-kit generate --custom intentionally creates no snapshot because it cannot infer the resulting schema state. drizzle-kit check passes; the chain is healthy.
- **Ruled out — FORCE RLS with no policies.** A naive extraction suggested 51 tables were RLS-forced without policies. A corrected extraction (policies use the form CREATE POLICY "..._isolation_policy" ON "table") proved a 1:1 match: every forced table has four policies (select/insert/update/delete) created in migration 0000 and partially revised in 0007.

2.3 Severity model

Findings use a four-level scale. **Critical** = exploitable with high impact or patch-available RCE in a pinned runtime dependency. **High** = exploitable vulnerability with meaningful preconditions, or supply-chain advisories on runtime paths. **Medium** = latent fragility, configuration drift, or hygiene debt that raises incident probability. **Low** = cosmetic, compliance-quality, or housekeeping items. Each finding carries an identifier (C1, H1-H6, M1-M9, L1-L6) used for cross-referencing throughout the execution plan and the remediation kit.

3. System Overview and Architecture

3.1 Stack and structure

Oxenn is a brand-intelligence platform (crawler, AI analysis, dashboards) implemented as a single Next.js 16 application with 358 TypeScript and 125 TSX source files. The src/ tree separates app routes ([locale] marketing pages, dashboard feature pages, api/v1 endpoints, server actions), features/ (domain modules such as admin, ai-intelligence, acquisition, billing), core/ (database context, cache, config, container), services/ (auth session, audit engine, monitoring), and components/. A second top-level components/ and lib/ tree exists at the repository root; Chapter 7 documents that all of it except one file is dead code.

The database layer lives in database/ (schema definitions, 13 Drizzle migrations with snapshots, bootstrap.sql, seed) with runtime tooling in src/core/database/ (migrator.ts, tenant-context, privileged-paths, tenant-tables.generated.ts, unit-of-work) and operational scripts in scripts/database/ (db-push-guard.ts, verify-rls-migrations.ts, verify-tenant-tables.ts, build-reference-matrix.ts). CI (.github/workflows/ci.yml) runs typecheck, lint, tenant-tables alignment, RLS coverage verification, the production build, a generated-files drift check, and the unit and UI test projects on every push to main.

3.2 Tenant isolation design

The tenant model is PostgreSQL-native row-level security. Every tenant-scoped table carries four per-operation policies keyed on NULLIF(current_setting('app.current_tenant_id', true), '')::uuid against either an organization_id or tenant_id column. Application code never sets the GUC directly; TenantContextManager.runWithTenantContext opens a transaction, applies set_config('app.current_tenant_id', $1, true) (parameterized), and runs the work under AsyncLocalStorage. Privileged operations (logins, admin probes) must run under explicitly registered purpose tags in runWithSystemContext, which leases a separate client from an authenticator pool, writes an audit_records row, and blocks test-only tags outside test environments. A generated allowlist (tenant-tables.generated.ts) plus a SQL tokenizer (isQueryTenantScoped) prevents system clients from touching tenant tables outside system mode. This is a substantially stronger design than typical application-layer-only filtering.

3.3 Migration chain inventory

The chain spans 13 migrations (0000-0012). 0000 creates the base schema — 167 policy statements across roughly 42 tenant tables plus global tables (users, roles, admin_users, audit_records, ai_engines, feature_flags, join tables). 0001-0003 add columns and policies; 0004 adds the document_embeddings HNSW index; 0005 is the hand-written FORCE RLS sweep across exactly 51 tables; 0006 adds foreign keys; 0007 re-aligns 12 policies; 0009-0010 introduce the session and email-verification tables; 0011 adds auth_rate_limits; 0012 normalizes user emails to lowercase behind a duplicate-check guard before creating a case-insensitive unique index — a correct and commendable data-migration pattern.

4. Critical Findings

C1 — Next.js 16.2.11: three CRITICAL RCE advisories, fix requires >= 16.3.6

Evidence. package.json pins "next": "16.2.11" (exact); pnpm-lock.yaml resolves next@16.2.11 with eslint-config-next@16.2.11. pnpm audit --json (2026-10-19) returns eight advisory records for the next package, three of them CRITICAL:

```
[CRITICAL] next <>=16.0.0 <16.3.3>  Unauthenticated Remote Code Execution
           on windows-hosted server          patched: >=16.3.3
[CRITICAL] next <>=16.0.0 <16.3.3>  RCE in Image Optimization
           (attacker-controlled cache key)   patched: >=16.3.3
[CRITICAL] next <>=16.2.0 <16.3.6>  RCE in next/og ImageResponse
                                             patched: >=16.3.6
```

Impact. The image-optimization and ImageResponse vectors apply to standard production deployments (the build output confirms next/image usage across dashboard routes), giving unauthenticated attackers a code-execution path in the server runtime. The Windows-dev-server vector is lower risk for Linux CI but relevant to any developer on Windows. Because these are RCE-class issues in the framework itself and patched versions exist, this outranks every other finding despite the otherwise healthy codebase.

Remediation. Bump next and eslint-config-next to **16.4.2** (current stable; satisfies all three floors). The minor-version hop (16.2 to 16.4) is low-risk but not zero: exercise image routes, next/og callers (invoice pages), and middleware behavior after upgrade. eslint-config-next must move in lockstep. Regression gates: pnpm typecheck, pnpm lint, both test projects, pnpm build, then pnpm audit must show zero critical advisories. Also verify that the three bundled esbuild versions no longer include 0.18.20 (see M3) after the lockfile update.

5. High-Severity Findings

H1 — Stored XSS: unsanitized LLM output injected via dangerouslySetInnerHTML

Evidence. src/components/features/analytics/LlmAnalyticsPanel.tsx, highlightMentions() (lines ≈218-234). The function receives LLM response text, wraps brand and competitor mentions in span markup, and hands the result to dangerouslySetInnerHTML without any escaping or sanitization:

```
const highlightMentions = (text: string) => {
  if (!text) return "";
  let highlighted = text;                        // raw, unescaped
  const brandRegex = new RegExp(`(${brandName})`, "g");   // unescaped needle
  highlighted = highlighted.replace(brandRegex,
    `<span class="...emerald...">$1</span>`);
  competitors.forEach((comp) => {
    const compRegex = new RegExp(`(${comp})`, "g");      // unescaped needle
    highlighted = highlighted.replace(compRegex,
      `<span class="...amber...">$1</span>`);
  });
  return <div dangerouslySetInnerHTML={{ __html: highlighted }} ... />;
};
```

Impact. The analytics response text is model-generated content about crawled third-party web pages — attacker-influenceable data by construction (a hostile page can plant text that survives crawling and shapes the LLM answer). Any HTML payload in that text renders inside an authenticated dashboard session, enabling cookie-adjacent attacks and action forgery under the victim's identity. A secondary defect: competitor names and the brand name are interpolated into RegExp unescaped, so names containing metacharacters throw SyntaxError and crash the panel render.

Contrast case (already correct). The documentation renderer at src/app/[locale]/docs/[slug]/page.tsx builds inline HTML via parseInlineMarkdown but terminates in DOMPurify.sanitize(text, { USE_PROFILES: { html: true } }) and the content source is first-party. It still warrants an SSR caveat (DOMPurify is a browser API; during server rendering sanitize is a pass-through), but the content trust boundary makes that LOW risk. The pattern to copy is that file's sanitize-before-inject discipline.

Remediation. Escape the response text first (HTML entities), escape both regex needles (HTML-escape then regex-escape, case-insensitive), and sanitize the final markup with DOMPurify as a second net. The remediation kit ships a complete drop-in replacement function (PATCH-LlmAnalyticsPanel-XSS.md) plus a regression-test spec asserting that a payload such as an img tag with an onerror handler survives as inert text. Effort is under one hour including tests.

H2 — undici 7.29.0: ten advisories on the AI runtime path

Evidence. pnpm audit reports ten undici advisories, worst-severity HIGH: TLS certificate validation bypass via dropped connections, multiple denial-of-service vectors (unhandled WebSocket errors, orphaned RetryHandler responses, unbounded decompression), cross-user cookie disclosure via Set-Cookie caching, response splitting and truncation. The lockfile pins undici 7.29.0, reached at runtime through @ai-sdk/provider-utils@5.0.29 — i.e., on every AI provider call.

Remediation. Not directly bumpable (transitive). Add a pnpm override "undici": ">=7.29.1 <8" and re-lock; the override stays inside the same major and matches the patched floor. Verify @ai-sdk/provider-utils behavior under the new undici with the existing AI unit tests and one live mock-crawler request.

H3 — axios < 1.20.0: twelve advisories via the crawler dependency

Evidence. Twelve axios advisories, HIGH worst severity: ReDoS in fromDataURI and shouldBypassProxy, prototype-pollution gadgets in fetch adapter and toFormData, HTTP/2 adapter bypassing DNS/proxy controls, header injection via inherited headers. The vulnerable chain sits under @mendable/firecrawl-js@4.35.0 — the external-content crawler, which is precisely the component with the least-trusted input.

Remediation. pnpm override "axios": ">=1.20.0 <2". Firecrawl-js declares broad axios 1.x compatibility; run the acquisition unit project plus one manual crawl against a mock target to confirm no behavioral drift.

H4 — sharp < 0.35.4: libvips and libheif CVEs via Next.js

Evidence. pnpm audit: HIGH — sharp inherits libvips CVE-2026-33327 (and a companion advisory) below 0.35.0, and libheif GHSA-g89c-p67h-r497 / GHSA-2jg2-4 below 0.35.4. sharp@0.34.5 is present transitively (Next.js image optimization) and appears in pnpm's ignored-build-scripts warning. — **Remediation.** The C1 upgrade to next 16.4.2 refreshes the bundled sharp floor; confirm with pnpm why/audit after re-locking, and include sharp in the pnpm onlyBuiltDependencies allowlist (M5) so its prebuilt binaries install deterministically.

H5 — eslint 9.39.5 deprecated and end-of-life

Evidence. The lockfile itself records the deprecation for eslint@9.39.5: “This version is no longer supported. Please see https://eslint.org/version-support”; the registry's current line is 10.13.0, and the project pins ^9.39.5. An EOL linter receives no security or bug fixes and its plugin ecosystem is migrating. — **Remediation.** Move to eslint ^10.13.0 together with eslint-config-next 16.4.2. The existing flat config (defineConfig([...nextVitals, ...nextTs], globalIgnores)) is v10-compatible in structure. If any rule churn appears, gate the lint job with --max-warnings=218 temporarily (matching today's count) and ratchet downward; do not let the upgrade block the security releases.

H6 — js-yaml < 4.3.2 and @grpc/grpc-js < 1.14.5 (transitive HIGHs)

Evidence. pnpm audit: js-yaml < 4.3.2 — CPU DoS via unlimited empty merge sources (tooling chain); @grpc/grpc-js 1.14.0-1.14.4 — getAuthContext can return unauthenticated context in certain configurations plus a lower-severity error-message exposure (inngest's gRPC transport chain). — **Remediation.** pnpm overrides "js-yaml": ">=4.3.2 <5" and "@grpc/grpc-js": ">=1.14.5 <2"; both are patch-in-range and need no code changes. Re-run the full test pair after re-lock.

6. Medium-Severity Findings

M1 — @types/node ^20 against engines node >=22 and CI Node 22

Evidence. package.json declares "engines": { "node": ">=22" } and CI installs Node 22 (.github/workflows/ci.yml, node-version: 22), while devDependencies pin "@types/node": "^20" (lockfile resolves 20.19.43). Local development on Node v24.21.0 widens the gap. **Impact.** Node 22/24 APIs are either untyped or typed against the wrong surface, inviting unsafe casts; type-level and runtime behavior can silently diverge between machines. **Remediation.** Set "@types/node": "^22" so types match the minimum supported runtime; a one-line change included in the remediation kit's package.json.

M2 — @types/dompurify deprecated stub; dompurify below patched floor

Evidence. Lockfile: "@types/dompurify@3.2.0 — deprecated: This is a stub types definition. dompurify provides its own type definitions, so you do not need this installed." Separately, pnpm audit rates dompurify 3.4.14 LOW (afterSanitize hook can leave detached nodes; patched >= 3.4.16; registry offers 3.4.17). **Remediation.** Remove @types/dompurify entirely and bump dompurify to ^3.4.17 — both in the kit. H1's fix introduces a new DOMPurify import in LlmAnalyticsPanel, so the self-shipped types become load-bearing.

M3 — drizzle-kit 0.31.10 pulls the deprecated @esbuild-kit chain

Evidence. drizzle-kit@0.31.10 depends on @esbuild-kit/esm-loader@2.6.5 and drags esbuild@0.18.20 through @esbuild-kit/core-utils@3.3.2. Both @esbuild-kit packages are marked deprecated in the lockfile (“Merged into tsx”), and esbuild 0.18.20 falls in the MODERATE advisory range <= 0.24.2 (development-server request forgery). Two newer esbuild copies (0.25.12 via drizzle-kit, 0.28.2 via vite) are healthy; the vulnerable one is dev-only. **Remediation.** Bump drizzle-kit to ^0.31.11 now; schedule the drizzle-kit 1.x migration (which replaced @esbuild-kit with tsx) as a planned task with a drizzle-kit generate dry-run diff review, since 1.x also changes some config semantics.

M4 — Mixed package-manager invocations inside a pinned-pnpm project

Evidence. packageManager is pnpm@10.30.3, yet scripts read "build": "npx tsx scripts/generate-docs-data.ts && next build" (npx consults the npm registry and can fetch a different tsx than the lockfile pins) and "docs": "npm run docs:matrix" (invokes npm inside a pnpm project). **Impact.** Non-hermetic builds; cold-start latency; divergence risk in air-gapped or CI-cached environments. **Remediation.** "build": "pnpm exec tsx ...", "docs": "pnpm run docs:matrix" — included in the kit.

M5 — pnpm 10 blocked build scripts are undeclared

Evidence. pnpm install emits: “Ignored build scripts: esbuild@0.18.20, esbuild@0.25.12, esbuild@0.28.2, protobufjs@7.6.5, sharp@0.34.5, unrs-resolver@1.12.2. Run pnpm approve-builds ...”. package.json contains no pnpm.onlyBuiltDependencies. **Impact.** Native/install hooks silently differ per machine; sharp may skip binary setup on fresh clones, producing slow first renders or odd failures that appear only outside CI. **Remediation.** Declare the allowlist in package.json (esbuild, sharp, protobufjs, unrs-resolver) so behavior is deterministic everywhere; shipped in the kit.

M6 — .env.example drift and weak env-check coverage

Evidence. Code reads AUTHENTICATOR_DATABASE_URL (src/features/admin/infrastructure/persistence/postgres/index.ts line 86), AUTHENTICATOR_POOL_MAX, EMAIL_PROVIDER, APP_MODE and VERIFICATION_PROBE_ACTIVE; none appear in .env.example. env-check.ts validates only seven variables (RULES, lines 14-64) and its SESSION_SECRET rule (16+ chars) accepts the well-known placeholder value shipped in .env.example — a validator that passes on a known insecure default. **Remediation.** Extend .env.example with the missing variables (shipped) and add a known-default rejection rule plus AUTHENTICATOR_* format checks to env-check; document bootstrap.sql as the required superuser step for pgvector.

M7 — Broken shadcn config and orphan root component trees

Evidence. components.json points aliases.utils at @/lib/utils, but src/lib/utils.ts does not exist (src/lib contains audit-engine, docsData.ts, docsIndex.ts, firecrawl.ts, inngest), and its tailwind.css value "app/globals.css" is wrong — the real file is src/app/globals.css. The cn() helper lives at the repository root (lib/utils.ts) and is imported only by the root components/ui/button.tsx, which nothing imports; components/live-analytics/ (8 files) is likewise unreferenced. The single live root file — components/navigation/FloatingSidebar.tsx — is imported via a fragile "../../../components/navigation/FloatingSidebar" relative path in src/app/[locale]/layout.tsx line 7, while sibling imports use the @/components/ alias, and an identically named directory exists under src/components/navigation/. **Remediation.** Create src/lib/utils.ts, correct components.json, git mv FloatingSidebar into src/components/navigation and switch the layout import to the alias, then delete the orphan trees (components/ui, components/live-analytics, lib/) after a final reference grep; exact commands appear in Chapter 10.

M8 — next.config.ts empty: no security headers, no hardening

Evidence. next.config.ts is the seven-line default with zero options; src/proxy.ts only redirects "/" to "/fa"; a repository-wide search finds no headers() configuration, no CSP, and poweredByHeader remains enabled (default). **Impact.** The X-Powered-By header advertises the stack; absence of X-Frame-Options, X-Content-Type-Options, Referrer-Policy and HSTS leaves the application relying on browser defaults for clickjacking and MIME-sniffing protection. **Remediation.** The kit ships a hardened next.config.ts with poweredByHeader: false and a baseline header set; adopt CSP in report-only mode as a follow-up so Next.js inline scripts are measured before enforcement.

M9 — Test coverage gaps and warning-tolerant gates

Evidence. vitest.config.ts runs the db project with passWithNoTests: true ("TEMP: flipped to false by OX-104 when tests/db is seeded") and tests/db contains no test files; the UI project comprises a single file with three tests against 125 TSX components; CI runs pnpm lint without a --max-warnings ceiling, so the current 218 warnings cannot regress the build. **Remediation.** Seed tests/db (closing OX-104), add UI coverage to the panels that render external content — LlmAnalyticsPanel first (the H1 regression test) — and add --max-warnings=218 as a ratchet that only moves downward.

7. Low-Severity and Hygiene Findings

L1 — Committed junk and stale artifacts

Evidence. database/schema/index.ts.tmp2 — a 1,175-line stale copy of the schema (diff shows it predates the auth tables: no sessions, no email_verification_tokens, no password_reset_tokens, no idx_users_lower_email). test_db.ts — an 82-byte scratch file whose body is console.log(typeof drizzle). Seven log files are tracked under verification/phase-0/ (npm-build.log, npm-lint.log, npm-tsc.log, pnpm-build.log, pnpm-lint.log, pnpm-tsc.log, replay-stock-failure.log) despite .gitignore's *.log rule — meaning they were committed before the ignore rule landed and never untracked. update_tenant.js is a one-off CommonJS script that mutated tenant-context source by regex; its effect is already in the code. get_routes.sh and get_signatures.sh are ad-hoc dev probes at the root. **Remediation.** git rm --cached plus delete (exact commands in Chapter 10); move the two dev scripts under scripts/dev-tools/ if they are still useful.

L2 — Migration file lacks trailing newline

Evidence. database/drizzle/0004_abandoned_black_crow.sql ends without a newline (117 bytes, wc -l = 0). Harmless to drizzle-kit and PostgreSQL but it confuses line-based tooling (this audit's own first hypothesis was born exactly there) and POSIX text conventions. **Remediation.** The kit ships the same file with a final newline; adopt .editorconfig with insert_final_newline = true.

L3 — Inngest scaffolding and rename leftovers

Evidence. src/lib/inngest/client.ts constructs new Inngest({ id: "seorchable" }) under the oxenn brand (the project was renamed — verification/phase-0/env.txt records a commit “Rename Seorchable to oxenn in SPEC.md”); functions.ts registers only hello-world and a placeholder scheduled-monitoring function; AGENTS.md is still titled “Seorchable — Agent Operating Contract”. **Remediation.** Rename the client id when monitoring goes live; sweep remaining seorchable strings; wire INNGEST_SIGNING_KEY (declared in .env.example, consumed by the SDK) before exposing /api/inngest publicly.

L4 — tsconfig target ES2017 for a Node >=22 codebase

Evidence. tsconfig.json sets "target": "ES2017" while engines demand Node 22+ and the dev environment runs Node 24. Downleveled emit can mask modern-API usage and forces transpilation of constructs the runtime supports natively. **Remediation.** Raise target to ES2022 (kit ships the file); typecheck remains clean and verified after the change.

L5 — Audit-log records carry placeholder identity fields

Evidence. The system-context audit writer (src/core/database/tenant-context/index.ts, doAuditLog) hardcodes actor_email “system@oxenn.local”, ip_address “0.0.0.0” and user_agent “system-context”, and swallows write failures with a console.error. **Impact.** Compliance-quality degradation: privileged access records cannot be attributed to real actors, and failed audit writes are silent. **Remediation.** Propagate the actor metadata already present on the context (userId), thread request metadata through, and add a retry or alerting path on audit-write failure.

L6 — pgvector extension not encoded in the migration chain

Evidence. Migration 0000 creates vector(768) columns, but extension creation happens outside the chain: migrator.ts (lines 22-37) runs CREATE EXTENSION IF NOT EXISTS vector before migrate() with a privilege-error fallback instructing the operator to run database/bootstrap.sql as superuser — a sound design for the db:migrate path, but drizzle-kit push (db:push) bypasses the migrator entirely and the db-push-guard does not check extension presence. **Remediation.** Document bootstrap.sql in the deployment README and add an extension-existence probe to db-push-guard.

8. Verified Strengths (Evidence-Based)

An audit is only credible if it also records what was checked and found sound. The following properties were verified positively and need no remediation; they should be protected as invariants during the fixes in Chapter 9.

- **Manifest-lockfile integrity.** pnpm install --frozen-lockfile completes in 11.4 s — package.json and pnpm-lock.yaml are in perfect sync (2,869 packages).
- **Type safety.** tsc --noEmit reports zero errors across 358 TS + 125 TSX files.
- **Migration chain integrity.** drizzle-kit check passes for all 13 journal entries; every snapshot's id sequence links correctly; the lone custom migration (0005) is journal-consistent by design.
- **RLS completeness.** Cross-checking the 51 FORCE-RLS tables in 0005 against policy creation in 0000 and 0007 yields an exact 1:1 match — four policies per table, zero gaps, zero orphans.
- **Parameterized SQL in privileged paths.** auth/session/account server actions use $1..$n placeholders throughout; tenant GUC is set via set_config($1); no sql.raw usage anywhere in src/.
- **Guardrail tooling.** db-push-guard implements an eight-stage production protection chain (environment validation, production markers, managed-host detection in DATABASE_URL, double opt-in flags, catalog emptiness check); verify-rls-migrations parses migration SQL with comment/string-stripping to prove RLS coverage; env-check gates migrations.
- **CI depth.** The workflow verifies tenant-tables generation drift (git diff --exit-code on generated files) — a check most teams never add.
- **Safe data migration pattern.** 0012 aborts when case-insensitive duplicate emails exist, normalizes, then creates the unique index — correct ordering.
- **Test health.** 164 unit tests and all UI tests pass; the production build completes and emits the full route manifest.
- **No hardcoded secrets.** Key-pattern scanning (AWS, Google, OpenAI-style, GitHub tokens, PEM blocks) found nothing across src/, scripts/, database/, components/, lib/; verification/env.txt contains only git metadata.

9. Execution Plan — Phased Roadmap

The plan is ordered so that every phase ends in a green, shippable state: safety first, then the critical dependency surgery, then code-level security, then toolchain hygiene and hardening. Each phase lists its covered findings, effort, validation gates (the same commands CI already runs, so nothing here is novel process), and rollback strategy. Effort estimates assume one developer familiar with the codebase.

Phase 0 — Safety net (before any change)

Phase 0 tasks

#	Task	Effort
0.1	Create branch fix/audit-remediation from a clean main; confirm CI green on main before starting.	5 min
0.2	Record baseline evidence: pnpm audit --json, pnpm lint output (218 warnings), test counts — the regression yardstick.	10 min
0.3	Verify a restorable backup exists for any shared database (no schema changes are planned; belt and braces).	10 min

Phase 1 — Dependency surgery (C1, H2-H6, M1-M3, M5)

Apply the remediation kit's package.json: next and eslint-config-next to 16.4.2, dompurify to ^3.4.17, @types/node to ^22, drizzle-kit to ^0.31.11, eslint to ^10.13.0, remove @types/dompurify, add the four pnpm overrides (undici, axios, js-yaml, @grpc/grpc-js) and the onlyBuiltDependencies allowlist, and normalize scripts to pnpm exec/run. Then pnpm install and reconcile the lockfile.

Phase 1 validation gates

Gate	Expectation
pnpm install	completes; overrides resolved; no peer-dep errors
pnpm audit	0 critical, 0 high (moderate esbuild note may remain until drizzle-kit 1.x)
pnpm typecheck	0 errors
pnpm test:unit / test:ui	164 + 3 passing, none skipped
pnpm build	succeeds; spot-check image routes and /api/inngest
pnpm lint	<= 218 warnings (eslint 10 may shift counts; ratchet down later)

Rollback. Single-commit dependency bump — git revert restores the exact prior lockfile. If an override provokes a peer conflict (most plausible: axios under firecrawl-js), loosen that single override to the nearest

compatible floor rather than dropping it. **Do not** combine Phase 1 with code changes; keep the diff reviewable.

Phase 2 — Code security (H1, M8)

Apply PATCH-LlmAnalyticsPanel-XSS.md (escape-then-highlight-then-sanitize, case-insensitive needles, DOMPurify import) and add the specified UI regression test. Land the hardened next.config.ts (poweredByHeader false + baseline headers). Run pnpm test:ui, pnpm typecheck, pnpm lint, pnpm build. Manually verify the analytics panel with a mocked response containing an img-onerror payload and a competitor name with regex metacharacters. Rollback: two-file revert, no data touchpoints.

Phase 3 — Repository and configuration hygiene (M6, M7, M9, L1, L2, L4)

Execute the deletion list (tmp2, test_db.ts, update_tenant.js, tracked logs; relocate FloatingSidebar; remove orphan root trees), replace components.json, add src/lib/utils.ts, update .env.example, ship the newline-corrected 0004 file, raise tsconfig target to ES2022, and add the lint --max-warnings ratchet to CI. Gates: full local gate set + CI green including the generated-files drift check (the FloatingSidebar move must not affect docsIndex or tenant-tables generated files).

Phase 4 — Scheduled follow-ups (separate change window)

- drizzle-kit 1.x migration (finishes M3): review generate dry-run diff, adopt new config semantics, remove esbuild 0.18.20 from the tree.
- CSP report-only rollout for M8 hardening; collect violation reports for two weeks before enforcing.
- react 19.3.1, framer-motion 14, @vercel/analytics 2, typescript 7 evaluation — each behind its own CI cycle.
- OX-104: seed tests/db, flip passWithNoTests to false; expand UI coverage beyond 3 tests.
- env-check hardening (M6): reject known-default SESSION_SECRET, validate AUTHENTICATOR_* formats.
- Audit-log attribution (L5) and db-push-guard pgvector probe (L6).

9.3 Risk assessment of the plan

The only phase with real regression potential is Phase 1 (framework minor hop plus four overrides). Confidence is high because: the advisory floors are strictly below the target versions (16.3.6 is well below 16.4.2); overrides stay within declared major ranges; and the project's own CI already exercises typecheck, lint, both test projects, build, RLS coverage, and generated-file drift — an unusually complete gate set. Phases 2-4 are low-risk, reversible, and mostly additive. Total estimated effort: 0.5-1 day for Phases 0-3 combined, with Phase 1 accounting for roughly half.

10. Command-Level Checklist

Execute top to bottom from the repository root. Items marked GATE must pass before continuing. The kit folder oxenn-fixes/ is assumed to sit next to the repository.

```
# ---- Phase 0: safety -------------------------------------------------
git status && git checkout main && git pull
git checkout -b fix/audit-remediation
pnpm install --frozen-lockfile                        # GATE: baseline installs
pnpm audit --json > ../baseline-audit.json            # record baseline

# ---- Phase 1: dependency surgery -------------------------------------
cp ../oxenn-fixes/package.json package.json
pnpm install                                          # lockfile updates here
pnpm audit                                            # GATE: 0 critical, 0 high
pnpm typecheck                                        # GATE: 0 errors
pnpm test:unit && pnpm test:ui                        # GATE: 164 + 3 pass
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/oxenn \
SESSION_SECRET=local-dev-secret-0123456789abcdef pnpm build   # GATE

# ---- Phase 2: code security ------------------------------------------
# apply PATCH-LlmAnalyticsPanel-XSS.md by hand (import + function swap)
cp ../oxenn-fixes/next.config.ts next.config.ts
pnpm test:ui && pnpm typecheck && pnpm lint           # GATE

# ---- Phase 3: hygiene ------------------------------------------------
git rm --cached database/schema/index.ts.tmp2 && rm database/schema/index.ts.tmp2
git rm --cached test_db.ts && rm test_db.ts
git rm --cached update_tenant.js && rm update_tenant.js
git rm --cached verification/phase-0/npm-build.log \
  verification/phase-0/npm-lint.log \
  verification/phase-0/npm-tsc.log verification/phase-0/pnpm-build.log \
  verification/phase-0/pnpm-lint.log verification/phase-0/pnpm-tsc.log \
  verification/phase-0/replay-stock-failure.log
git mv components/navigation/FloatingSidebar.tsx src/components/navigation/
# edit src/app/[locale]/layout.tsx line 7:
#   import FloatingSidebar
#     from "@/components/navigation/FloatingSidebar";
git rm -r --cached components lib && rm -rf components lib
cp ../oxenn-fixes/components.json components.json
mkdir -p src/lib && cp ../oxenn-fixes/src/lib/utils.ts src/lib/utils.ts
cp ../oxenn-fixes/.env.example .env.example
cp ../oxenn-fixes/database/drizzle/0004_abandoned_black_crow.sql database/drizzle/
cp ../oxenn-fixes/tsconfig.json tsconfig.json
pnpm typecheck && pnpm test:unit && pnpm test:ui && pnpm build   # GATE

# ---- Final database verification -------------------------------------
npx drizzle-kit check                                 # GATE: chain consistent
pnpm db:verify-rls-migrations                         # GATE
pnpm tsx scripts/database/verify-tenant-tables.ts     # GATE

# ---- Ship ------------------------------------------------------------
git add -A && git commit -m "fix: audit remediation (C1,H1-H6,M1-M9,L1-L4)"
git push -u origin fix/audit-remediation              # open PR, watch CI
```

Appendix A — Tool Output Evidence

A.1 pnpm audit summary (2026-10-19)

pnpm audit --json returned 54 advisories across 12 distinct modules: 3 critical, 25 high, 19 moderate, 7 low. The donut chart in Chapter 1 shows the severity mix; the bar chart shows the per-module distribution with each module colored by its worst severity. The next module's 8 advisories include the 3 critical RCE records; axios (12) and undici (10) dominate the high-severity count through the crawler and AI SDK chains respectively.

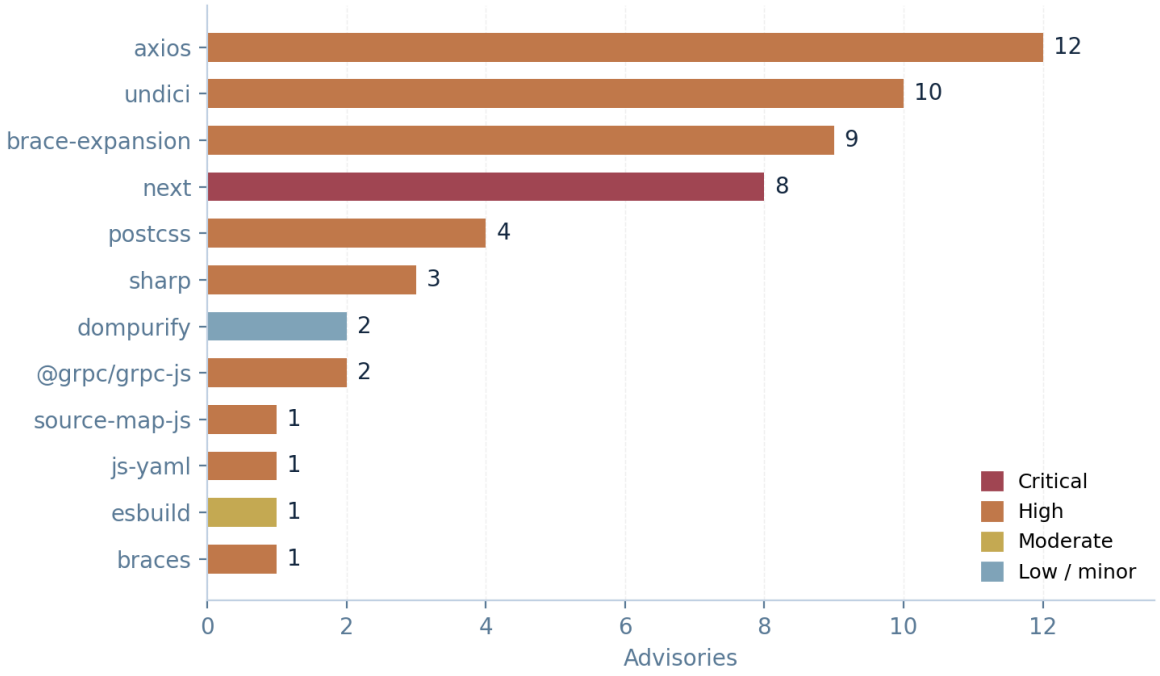


Figure 2 — Advisories per module, colored by worst severity (next includes the 3 critical RCE records).

A.2 Key lockfile deprecation records (verbatim)

```
eslint@9.39.5:
  deprecated: This version is no longer supported. Please see
              https://eslint.org/version-support for other options.

@types/dompurify@3.2.0:
  deprecated: This is a stub types definition. dompurify provides its
              own type definitions, so you do not need this installed.

@esbuild-kit/esm-loader@2.6.5:
@esbuild-kit/core-utils@3.3.2:
  deprecated: Merged into tsx: https://tsx.hirok.io
```

A.3 Test and build evidence

```
Test Files  41 passed | 1 skipped (42)     [unit]
     Tests  164 passed | 1 todo (165)
     Duration  13.91s

Test Files  1 passed (1)                   [ui]
     Tests  3 passed (3)

✖ 218 problems (0 errors, 218 warnings)    [eslint]

Everything's fine  [drizzle-kit check: journal + snapshots consistent]
Done in 11.4s     [pnpm install --frozen-lockfile, pnpm v10.30.3]
```

A.4 Environment facts

- Analyzed snapshot: oxenn-main (2).zip, 769 files, no node_modules included.
- Toolchain: Node v24.21.0, pnpm 10.30.3 (installed for this audit), npm 11.19.0.
- Registry state verified 2026-10-19: next latest 16.4.2; eslint latest 10.13.0; @types/node latest 26.6.5; drizzle-kit latest 0.31.11; dompurify latest 3.4.17; ai latest 7.0.143; @ai-sdk/google latest 4.0.99.
- AI SDK alignment check: ai@7.0.77 and @ai-sdk/google@4.0.50 both resolve @ai-sdk/provider@4.0.7 + @ai-sdk/provider-utils@5.0.29 — no version skew.

Appendix B — Dependency Upgrade Matrix

Target state after Phase 1

Package	Current	Target	Driver
next / eslint-config-next	16.2.11	16.4.2	C1: 3 critical RCE, floors 16.3.3 / 16.3.6
eslint	9.39.5 (EOL)	^10.13.0	H5: v9 deprecated (lockfile notice)
dompurify	^3.4.14	^3.4.17	M2: LOW advisories patched 3.4.16
@types/dompurify	^3.2.0	remove	M2: deprecated stub
@types/node	^20	^22	M1: match engines/CI Node 22
drizzle-kit	^0.31.10	^0.31.11	M3: interim patch; 1.x later
undici (override)	7.29.0	>=7.29.1 <8	H2: 10 advisories incl. TLS bypass
axios (override)	1.x < 1.20	>=1.20.0 <2	H3: 12 advisories
js-yaml (override)	<4.3.2	>=4.3.2 <5	H6: CPU DoS
@grpc/grpc-js (override)	<1.14.5	>=1.14.5 <2	H6: auth-context flaw
ai / @ai-sdk/google	7.0.77 / 4.0.50	^7.0.143 / ^4.0.99	routine minors, provider-aligned
react / react-dom	19.2.4	19.2.4 (hold)	defer 19.3.1 to Phase 4 window

Versions marked override are transitive and must be forced via pnpm.overrides; all others are direct manifest bumps. After Phase 1, re-run pnpm why undici and pnpm why axios to confirm single-version resolution — duplicate copies across overrides boundaries would reintroduce the vulnerable ranges.